

Problem Set 3

CMSC 27200: Theory of Algorithms

Problem 1: Recurrence Relations (30 points, 5 extra credit points available)

Solve each of the following recurrence relations with the given initial value(s). Show your work.

- (a) 10 points: $T(n) = T(\frac{n}{2}) + n, T(1) = 1$

Solution:

$$\begin{aligned} T(n) &= T\left(\frac{n}{2}\right) + n \\ &= T\left(\frac{n}{4}\right) + \frac{n}{2} + n \\ &= T\left(\frac{n}{8}\right) + \frac{n}{4} + \frac{n}{2} + n \\ &\vdots \\ &= T\left(\frac{n}{2^k}\right) + n + \frac{n}{2} + \frac{n}{4} + \dots + \frac{n}{2^{k-1}} \\ &= T\left(\frac{n}{2^k}\right) + n \sum_{i=0}^{k-1} \frac{1}{2^i} \\ &= T\left(\frac{n}{2^k}\right) + n \frac{\frac{1}{2^k} - 1}{\frac{1}{2} - 1} \\ &= T\left(\frac{n}{2^k}\right) + 2n(1 - 2^{-k}) \end{aligned}$$

Let $k = \log_2 n$, we have

$$T(n) = T(1) + 2n\left(1 - \frac{1}{n}\right) = 2n - 1.$$

- (b) 10 points: $T(n) = 4T(\frac{n}{2}) + 3n^2, T(1) = 3$

Solution:

$$\begin{aligned}
T(n) &= 4T\left(\frac{n}{2}\right) + 3n^2 \\
&= 4^2T\left(\frac{n}{4}\right) + 4 \cdot 3\left(\frac{n}{2}\right)^2 + 3n^2 \\
&= 4^3T\left(\frac{n}{8}\right) + 4^2 \cdot 3\left(\frac{n}{4}\right)^2 + 2 \cdot 3n^2 \\
&\vdots \\
&= 4^kT\left(\frac{n}{2^k}\right) + k \cdot 3n^2
\end{aligned}$$

Let $k = \log_2 n$, we have

$$T(n) = 4^{\log_2 n} T(1) + \log_2 n \cdot 3n^2 = 3n^2 + 3n^2 \log_2 n = 3n^2(1 + \log_2 n).$$

(c) 10 points: $T(n) = 4T\left(\frac{n}{2}\right) + n^2 + 3$, $T(1) = 2$

Solution:

$$\begin{aligned}
T(n) &= 4T\left(\frac{n}{2}\right) + n^2 + 3 \\
&= 4^2T\left(\frac{n}{4}\right) + 4\left(\frac{n}{2}\right)^2 + n^2 + 4 \cdot 3 + 3 \\
&= 4^3T\left(\frac{n}{8}\right) + 4^2\left(\frac{n}{4}\right)^2 + 2n^2 + (4^2 \cdot 3 + 4 \cdot 3 + 3) \\
&= 4^4T\left(\frac{n}{16}\right) + 4^3\left(\frac{n}{8}\right)^2 + 3n^2 + (4^3 \cdot 3 + 4^2 \cdot 3 + 4 \cdot 3 + 3) \\
&\vdots \\
&= 4^kT\left(\frac{n}{2^k}\right) + kn^2 + 3(1 + 4 + \dots + 4^{k-1}) \\
&= 4^kT\left(\frac{n}{2^k}\right) + kn^2 + (4^k - 1)
\end{aligned}$$

Let $k = \log_2 n$, we have

$$T(n) = 4^{\log_2 n} T(1) + n^2 \log_2 n + (4^{\log_2 n} - 1) = n^2 \log_2 n + 3n^2 - 1.$$

(d) 5 points extra credit: $T(n) = 6T\left(\frac{n}{2}\right) - 8T\left(\frac{n}{4}\right) + 6n + 5$, $T(1) = 1$, $T(2) = 6$

Solution: Consider the homogeneous part $T_0(n) = 6T_0\left(\frac{n}{2}\right) - 8T_0\left(\frac{n}{4}\right)$. Plugging in $T_0(n) = n^p$, we have $n^p = 6\frac{n^p}{2^p} - 8\frac{n^p}{4^p}$. Dividing both sides by n^p yields $1 - \frac{6}{2^p} + \frac{8}{2^{2p}} = 0$. Let $x = 2^{-p}$. Then $1 - 6x + 8x^2 = (1 - 4x)(1 - 2x) = 0$, so $x = \frac{1}{2}$ or $\frac{1}{4}$, which implies $p = 1$ or 2 . Thus, the vector space of solutions to the homogeneous recurrence is $\{An^2 + Bn : A, B \in \mathbb{R}\}$. Since n is a solution to the homogeneous part, we try $T_p(n) = an \log_2 n + b$ instead. Plugging it into the recurrence relation, we have

$$\begin{aligned}
an \log_2 n + b &= 6 \left(a \frac{n}{2} \log_2 \frac{n}{2} + b \right) - 8 \left(a \frac{n}{4} \log_2 \frac{n}{4} + b \right) + 6n + 5 \\
&= 6 \left(\frac{an}{2} \log_2 n - \frac{an}{2} + b \right) - 8 \left(\frac{an}{4} \log_2 n - \frac{an}{2} + b \right) + 6n + 5 \\
&= an \log_2 n + (-3a + 4a + 6)n + (6b - 8b + 5).
\end{aligned}$$

So we have $a = -6$ and $b = \frac{5}{3}$.

Therefore, the general solution is $T(n) = An^2 + Bn - 6n \log_2 n + \frac{5}{3}$. Since $T(1) = 1$ and $T(2) = 6$, we have $A + B + \frac{5}{3} = 1 \implies A + B = -\frac{2}{3}$ and $4A + 2B - 12 + \frac{5}{3} = 6 \implies 4A + 2B = \frac{49}{3}$. Solving this gives $A = \frac{53}{6}$ and $B = -\frac{19}{2}$. Thus, the solution is

$$T(n) = \frac{53}{6}n^2 - \frac{19}{2}n - 6n \log_2 n + \frac{5}{3}.$$

Problem 2: Finding the k th Element (20 points)

You are given an array A of length n and an array B of length m , both sorted in increasing order of their elements. Give an $O(\log(n) + \log(m))$ time algorithm to find the k th smallest element of the union of A and B . For example, if array A has the elements 1, 4, 8, 10, array B has the elements 3, 5, 6, 9, and $k = 6$, then your algorithm should return 8.

You may assume that A and B do not have any elements in common. You should prove that your algorithm works and argue that it indeed runs in $O(\log(n) + \log(m))$ time. Hint is available.

Solution:

`kth_element(A, B, k)`

1. If $|A| = 0$, return $B[k]$.
2. If $|B| = 0$, return $A[k]$.
3. Let $m_A = \lceil |A|/2 \rceil$ and $m_B = \lceil |B|/2 \rceil$.
 - (i) If $k < m_A + m_B$, then return `kth_element(A[1, ..., m_A - 1], B, k)` if $A[m_A] > B[m_B]$, or `kth_element(A, B[1, ..., m_B - 1], k)` otherwise.
 - (ii) If $k \geq m_A + m_B$, then return `kth_element(A, B[m_B + 1, ..., |B|], k - m_B)` if $A[m_A] > B[m_B]$, or `kth_element(A[m_A + 1, ..., |A|], B, k - m_A)` otherwise.

Correctness: We prove that if A and B are sorted arrays of length n and m , and $k \in [m + n]$, then the algorithm correctly returns the k th element of the union of A and B (denoted by $A \cup B$) by strong induction on $m + n$. The base cases are $m = 0$ or $n = 0$, in which case the algorithm executes either step 1 or 2, which gives the correct value by construction.

Now consider arbitrary $m + n$ where $m \geq 1$ and $n \geq 1$. If $k < m_A + m_B$, then the algorithm will get to step 3(i). It suffices to show that the elements discarded by the recursive call have higher rank than k in $A \cup B$. If $A[m_A] > B[m_B]$, since A and B are sorted, for $j \geq m_A$ we

have $A[j] \geq A[m_A] > A[i]$ for $i \in \{1, \dots, m_A - 1\}$ and $A[j] \geq A[m_A] > B[m_B] \geq B[i]$ for $i \in \{1, \dots, m_B\}$. Thus, there are at least $m_A + m_B - 1 \geq k$ many elements in $A \cup B$ that are less than $A[j]$, and hence the k th element in $A \cup B$ is the same as the k th element in $A[1, \dots, m_A - 1] \cup B$. Since $n > 1$, $\lceil n/2 \rceil - 1 < n$, and the correctness follows by applying the inductive hypothesis to the recursive call. The argument for the second case in step 3(i) is similar.

If $k \geq m_A + m_B$, then the algorithm will get to step 3(ii). If $A[m_A] > B[m_B]$, we claim that for $j \leq m_B$, the rank of $B[j]$ is less than k in $A \cup B$ and hence removing it will reduce the rank of the k th element in $A \cup B$ by one. We count the number of elements in $A \cup B$ that are less than $B[j]$. Since A and B are sorted, we have $B[j] < B[i]$ for $i > j$ and $B[j] \leq B[m_B] < A[m_A] \leq A[i]$ for $i \geq m_A$. There are at most $j - 1 + m_A - 1 \leq m_A + m_B - 2$ many elements in $A \cup B$ that are less than $B[j]$, and the rank of $B[j]$ is at most $m_A + m_B - 1 \leq k - 1$. So the $(k - m_B)$ th element in $A \cup B[m_B + 1, \dots, m]$ is the same as the k th element in $A \cup B$. Since $m \geq 1$, $m - \lceil m/2 \rceil < m$. By strong induction, the recursive call computes the $(k - m_B)$ th element in $A \cup B[m_B + 1, \dots, m]$ correctly. The case of $A[m_A] < B[m_B]$ is similar.

Efficiency: At each recursive call, we reduce the length of A or B by half. Moreover, the rest of the operations in step 3 take constant time. It takes $O(\log m + \log n)$ many recursive calls to reduce one of the arrays to have length 0, at which point it takes $O(1)$ time to output $A[k]$ or $B[k]$.

Problem 3: The Mafia Game (20 points)

A group of $n > 2$ friends are playing a game of Mafia. In each round of this game, the n players can select any subset of the n players to interrogate. Some number of the players are **Mafia**, and they will be revealed during the interrogation. Denote by $[n] = \{1, \dots, n\}$ the set of n players.

- (a) 10 points: Suppose that there is only one Mafia member among the n players. In this case, we may represent the interrogation as a function I that takes as input any non-empty subset $S \subseteq [n]$ and outputs a boolean in $\{\text{TRUE}, \text{FALSE}\}$ indicating whether the subset contains a Mafia member. More formally, if i is Mafia, then $i \in S \iff I(S) = \text{TRUE}$.

Provide a description (in words) of a divide-and-conquer procedure that allows the players to identify the mafia member in $O(\log n)$ interrogations.

You need to show that this procedure always finds the Mafia and argue that your procedure only takes $O(\log n)$ tests.

Solution: We can run a simple divide-and-conquer scheme by splitting P in half and testing half. For example, we may begin by testing the set $P = \{P_1, P_2, \dots, P_{n/2}\}$. If this returns TRUE then we have found a set of size $n/2$ with a mafia member, in which case we can recursively run the algorithm, or else the mafia member is in the other half so we can recurse on that half instead.

Correctness: Note that the Mafia member must either be in the set we interrogate or the set we don't interrogate, and our interrogation identifies which set the mafia is in and recurses on that set. As such, we call our algorithm on continuously decreasing sets with the mafia in it, so eventually we will find a set of size one containing the Mafia.

Efficiency: In each step, we are cutting the size of set of potential Mafia down by half, using one test. Thus, our algorithm follows the recurrence relation

$$T(n) = T\left(\frac{n}{2}\right) + 1,$$

which by the master theorem gives us $T(n) = \log(n)$.

- (b) 10 points: Suppose instead that there are two Mafia in the group of n players. Furthermore, they can cover each other's tracks in the interrogation: if both are in the same interrogation, then the players won't be able to detect that they are Mafia!

Formally, there exist two players $i \neq j$ (who are Mafia) such that

$$I(S) = \begin{cases} \text{FALSE} & \text{If neither } i \text{ nor } j \text{ are in } S, \\ \text{TRUE} & \text{If exactly one of } i \text{ or } j \text{ are in } S, \\ \text{FALSE} & \text{If both } i \text{ and } j \text{ are in } S. \end{cases}$$

Describe a procedure for selecting subsets to interrogate that allows the players to identify both Mafia in $O(\log n)$ interrogations. You may invoke your algorithm from part (a).

Sketch a proof that your procedure is correct and that it takes $O(\log n)$ interrogations.

Solution: Note that, whenever a test returns TRUE, we can simply call the algorithm from part (a). As such, if we find a sequence of $O(\log n)$ interrogations where we can guarantee that at least one is TRUE, then we are done.

There are many possibilities for potential sequences, but we will only present one of them. Our solution is this: on test i , partition the people into 2^i groups, and test every other group. For example, on test 1 we are testing the left half of the people, and on test 3 we are splitting the players into eights and testing the first, third, fifth, and seventh group. After $\log n$ tests, we will have split into n different sections, and if none of the other tests returned TRUE then we know this one will.

Correctness: We only need to show that our procedure outputs TRUE at some point. Note that interrogation 1 tells us both mafia members are in the same half, and test 2 tells us that they are both in the same half of one half (or same quarter). Overall, FALSE outputs for tests 1 through i tell us that the two mafia are in the same block of size $n/2^i$, so by test $\log n$, a result of FALSE would indicate that both mafia are in the same block of size 1, which is impossible.

Efficiency: Note that once we get a TRUE output, we may simply run the procedure from (a) to identify mafia members. We can see that we only need to run $\log n$ interrogations to ensure a TRUE output by the argument above. From there, we need to run the procedure from (a) twice, giving us $3 \log n = O(\log n)$ interrogations.

Problem 4: Rephrasing of Problem 5 in Chapter 5 of Kleinberg-Tardos (20 points)

Given n lines $\{y = a_i x + b_i : i \in [n]\}$, give an $O(n \log(n))$ time algorithm to find all of the lines which can be seen from above without passing through another line. In other words, find all $i \in [n]$ with the property that there exists $x \in \mathbb{R}$ such that $a_i x + b_i \geq a_j x + b_j$ for all $j \in [n] \setminus \{i\}$.

You should explain why your algorithm works and takes $O(n \log(n))$ time. Hint is available.

Solution: For simplicity, assume no three lines intersect at the same point. We sort the lines by their slopes, i.e., by increasing order of a_i . Suppose $a_1 a_2 \dots a_n$, assume without loss of generality $a_i \neq a_j$ for all $i \neq j$, since otherwise the line with smaller b value will be blocked by the other line and it will not be visible from above. These line can be removed in $O(n)$ time after sorting the slopes.

VisibleLine(S)

1. If $|S| = 1$, return S
2. Let $mid = \lceil |S|/2 \rceil$
3. $A = \text{VisibleLine}(L[1, \dots, mid])$ and let $n = |A|$
4. $B = \text{VisibleLine}(L[mid + 1, \dots, |S|])$ and let $m = |B|$
5. Compute an array L of length n such that $L[1] = \infty$ and $L[i]$ equals to the x -coordinate of the intersection between lines $A[i-1]$ and $A[i]$ for $i = 2, \dots, n$
6. Compute an array R of length m such that $R[m] = \infty$ and $R[i]$ equals to the x -coordinate of the intersection between lines $B[j]$ and $B[j+1]$ for $j = 1, \dots, m-1$
7. $i = j = 1$
8. While $i \leq n$ and $j \leq m$:
 - (i) If $L[i] \leq R[j]$ and line $A[i]$ is above line $B[j]$ at $x = L[i]$, $i = i + 1$
 - (ii) If $R[j] \leq L[i]$ and line $A[i]$ is above line $B[j]$ at $x = R[j]$, $j = j + 1$
 - (iii) Else, return $[A[1], \dots, A[i], B[j], \dots, B[m]]$

Correctness: It suffices to show that the combine step correctly computes the array of lines that are visible from above and they are sorted by slopes if the lines in A and B are sorted by slopes and all lines in A have less slope than any line in B . Assuming this, by strong inductive hypothesis the A and B computed in step 3 and 4 are correct, then the correctness of the combine step completes the inductive step.

Observe that the curve formed by the visible lines is a piecewise linear function with increasing slope. More precisely, it is the function $f(x) = \max_{i \in [n]} \{a_i x + b_i\}$. Let $f_A(x)$ and $f_B(x)$ be the curves formed by the visible lines in A and B respectively. We have $f(x) = \max\{f_A(x), f_B(x)\}$. Since both f_A and f_B are piecewise linear with increasing slopes and the slopes in f_B are greater than the slopes in f_A , once they intersect at x_0 , $f_B(x) \geq f_A(x)$ for all $x \geq x_0$, i.e., f_A is no longer

visible but f_B becomes visible for $x \geq x_0$.

In the combine step (steps 5 to 8), the algorithm goes over the intersections between adjacent lines in A and the intersections between adjacent lines in B in increasing order of x -coordinate. The algorithm returns at the first time it witnesses an intersection with x -coordinate x' such that $f_A(x') < f_B(x')$. Let x'' be the x -coordinate of the previous intersection, then we have $f_A(x'') \geq f_B(x'')$ but $f_A(x') < f_B(x')$. Thus, f_A intersects with f_B at some x_0 where $x'' \leq x_0 < x'$. Since $A[i]$ and $B[j]$ are the visible lines when f_A intersects with f_B , $A[1], \dots, A[i]$ are visible but none of the lines in B are visible for $x \leq x_0$. For $x > x_0$, the lines $B[j], \dots, B[m]$ are visible but none of the lines in A are visible. Thus, the set of visible lines is precisely $\{A[1], \dots, A[i], B[j], \dots, B[m]\}$, as desired.

Efficiency: The combine step (steps 5 to 8) takes $O(n)$ time in total and steps 1 to 2 takes $O(1)$ time. The running time $T(n)$ satisfies the recurrence relation $T(n) = 2T(n/2) + O(n)$, and the solution to this is $O(n \log n)$. In the preprocessing step we sort the lines by their slope, which takes $O(n \log n)$ time. So the total running time is still $O(n \log n)$.

Hint:

2. At each step, you should try and narrow down the range of elements to consider by half for at least one of the two arrays.

4. You can divide up the lines based on their slope. If you solve the problem for the subset of lines with smaller slopes and the subset of lines with larger slopes, can you combine your solutions to obtain a solution for the entire problem?